

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Регулярные выражения

Термин регулярные выражения (**regular expressions**) чаще всего в статьях пишут в сокращённом варианте **regex** или **regexр**.

В Python поддержка регулярных выражений реализована с помощью стандартного модуля **re**.

Регулярные выражения создают шаблон (паттерн) для поиска информации, чаще всего находящейся в тексте, с помощью синтаксиса основанного на метасимволах **. ^ \$ * + ? { } [] | ()** или с помощью методов.

Назначение метасимволов:

- **.** — Один любой символ, кроме новой строки **\n**
- **?** — 0 или 1 вхождение шаблона слева
- **+** — 1 и более вхождений шаблона слева
- ***** — 0 и более вхождений шаблона слева
- **\w** — Любая цифра или буква или подчеркивание, эквивалентно **[a-zA-Z0-9_]** (**\W** — все, кроме буквы или цифры и знака подчёркивания)
- **\d** — Любая цифра от 0 до 9 эквивалентно **[0-9]** (**\D** — все, кроме цифры)
- **\s** — Любой пробельный символ (**\S** — любой непробельный символ)
- **\b** — Граница слова
- **[...]** — Один из символов в скобках (**[^...]** — любой символ, кроме тех, что в скобках)
- **** — Экранирование специальных символов (**\.** означает точку или **\+** — знак «плюс»)
- **^** и **\$** — Начало и конец строки соответственно
- **{n,m}** — От **n** до **m** вхождений (**{n,}** — **n** и больше, **{,m}** — от 0 до **m**)
- **a|b** — Соответствует **a** или **b**
- **(...)** — Группирует выражение и возвращает найденный текст
- **\t, \n, \r** — Символ табуляции, новой строки и возврат каретки

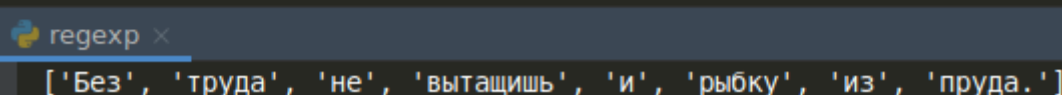
Основные методы, которые предоставляет библиотека **re**:

- **re.match(pattern, string, [flags=0])**
- **re.search(pattern, string, [flags=0])**
- **re.findall(pattern, string, [flags=0])**
- **re.split(pattern, string, [maxsplit=0], [flags=0])**
- **re.sub(pattern, replace, string, [count=0], [flags=0])**
- **re.compile(pattern, [flags=0])**

Метод **re.split()**

```
import re

text = 'Без труда не вытащишь и рыбку из пруда.'
print(re.split('\s', text))
```



Разделяет текст по пробелам. Но при таком написании пайчарм ругается и выдаёт предупреждение **invalid escape sequence**. Чтобы этого избежать, нужно использовать необработанные (сырые) строки – пометить наше выражение с помощью **r**.

```
import re

text = 'Без труда не вытащишь и рыбку из пруда.'
print(re.split(r'\s', text))
```



regex x

```
['Без', 'труда', 'не', 'вытащишь', 'и', 'рыбку', 'из', 'пруда.']
```

Метод `re.sub()`

Метод ищет шаблон в строке, и заменяет его на указанную подстроку. Если шаблон не найден, строка остается неизменной.

```
import re

text = 'Мой любимый цвет - зелёный.'
print(re.sub('зелёный', 'синий', text))
```

regex x

```
Мой любимый цвет - синий.
```

Метод `re.match()`

Этот метод ищет по заданному шаблону в начале строки. Сразу вывести на печать не получится, мы получим объект.

```
import re

text = 'Time is money'
print(re.match('Time', text))
```

regex x

```
<re.Match object; span=(0, 4), match='Time'>
```

И чтобы получить вывод в консоли, нужно использовать метод **group()**.

```
import re

text = 'Time is money'
res = re.match('Time', text)
print(res.group())
```

regex x
Time

Если же то что вы ищете отсутствует, или находится не в начале строки, функция вернёт None.

```
import re

text = 'Time is money'
res = re.match('money', text)
print(res)
```

regex x
None

Также есть методы `start()` и `end()` для того, чтобы узнать начальную и конечную позицию найденной подстроки. То есть Time занимает часть строки с нулевого индекса по четвёртый. Здесь как и в срезах последнее число не учитывается.

```
import re

text = 'Time is money'
res = re.match('Time', text)
print(res.start())
print(res.end())
```

regex x
0
4

Метод `re.search()`

Аналогичен `match()`, но ищет не только в начале строки, а по всей строке. В случае, если вхождений в строке несколько, выдаст первое вхождение.

```
import re

text = 'Time is money'
res = re.search('money', text)
print(res.group())
```

regex x
money

Метод `re.findall()`

Этот метод возвращает список всех найденных совпадений.

```
import re

text = 'Лето, ах лето, лето звёздное будь со мной!'
res = re.findall('лето', text)
print(res)
```

regex x
['лето', 'лето']

Метод `re.compile()`

Этот метод позволяет заранее скомпилировать регулярное выражение, а затем использовать его. Метод `re.compile()` возвращает объект `RegexObject`, имеющий также свои методы.

```
import re

text = 'Хищники: лиса, волк, рысь.'
res = re.compile('волк')
print(res.findall(text))
```

regex x
['волк']

Таким образом, поместив шаблон поиска в переменную, мы получаем больше гибкости в использовании. То есть мы можем работать с этой переменной применяя различные методы.

```
import re

text = 'Хищники: лиса, волк, рысь.'
res = re.compile('волк')
result = re.search(res, text)
print(result.group())
```

regex x

ВОЛК

А так мы можем посмотреть какие методы доступны у объекта RegexObject.

```
import re

text = 'Хищники: лиса, волк, рысь.'
res = re.compile('волк')
result = re.search(res, text)
for i in dir(res):
    if not i.startswith('_'):
        print(i)
```

regex x

flags
fullmatch
groupindex
groups
match
pattern
scanner
search
split
sub
subn

Как видно часть методов совпадает с теми что уже есть в модуле re, то есть мы можем их применять прямо к объекту Regex (здесь res).


```
import re

text = 'Хищники: лиса, волк, рысь.'
res = re.compile('волк')
result = res.search(text)
print(result.group())
```

regex x

волк

```
import re

text = 'Хищники: лиса, волк, рысь.'
res = re.compile('волк')
print(res.sub('медведь', text))
```

regex x

Хищники: лиса, медведь, рысь.

Примеры использования метасимволов

- — Один любой символ, кроме новой строки `\n`

```
import re

text = 'Жить хорошо, а хорошо жить ещё лучше!'
res = re.search('.o.o.o', text)
print(res.group())
```

regex x

хорошо

- ? — 0 или 1 вхождение шаблона слева, синоним `{0,1}`
- + — 1 и более вхождений шаблона слева, синоним `{1,}`
- * — 0 и более вхождений шаблона слева, синоним `{0,}`

Практически одинаковые по назначению, отличие только количеством вхождений символов.

```
import re

text = 'Какой антивирус выбрать - платный или бесплатный?'
res = re.findall('плата{0,1}', text)
print(res)
```

```
regex ×
['плат', 'плат']
```

```
import re

text = 'Абракадабра'
res = re.findall('бра+', text)
print(res)
```

```
regex ×
['бра', 'бра']
```

```
import re

text = 'Pentest или Penetration test – тест на проникновение'
res = re.findall('Pena*', text)
print(res)
```

```
regex ×
['Pen', 'Pen']
```

\w — Любая цифра или буква или нижнее подчёркивание (**\W** — все, кроме буквы или цифры и знака подчёркивания)

```
import re

text = 'Cross-Site Scripting'
res = re.findall(r'S\w', text)
print(res)
```

```
regex ×
['Si', 'Sc']
```



```
import re

text = 'Cross-Site Scripting'
res = re.findall(r's\W', text)
print(res)
```

regex ×

['s-']

\d — Любая цифра от 0 до 9 (**\D** — все, кроме цифры)

```
import re

text = 'C 10.00 до 20.00'
res1 = re.findall(r'\D', text)
res2 = re.findall(r'\d', text)
print(res1, res2, sep='\n')
```

regex ×

['C', ' ', '.', ' ', 'д', 'о', ' ', '.']
 ['1', '0', '0', '0', '2', '0', '0', '0']

\s — Любой пробельный символ (**\S** — любой непробельный символ)

```
import re

text = 'Что такое XSS?'
res1 = re.findall(r'ta\S', text)
res2 = re.findall(r'\s', text)
print(res1, res2, sep='\n')
```

regex ×

['так']
 [' ', ' ', ' ']

\b — Граница слова (начало или конец слова)

```
import re

text = 'Спасибо вам большое!'
res = re.findall(r'\bвам', text)
print(res)
```

regex ×

['вам']

[...] — Один из символов в скобках (**[^...]** — любой символ, кроме тех, что в скобках)

```
import re

text = 'Сказал "а" говори и "б"'
res = re.findall(r'[аб]', text)
print(res)
```

```
regex x
['а', 'а', 'а', 'б']
```

```
import re

text = 'Python'
res = re.findall(r'^yo', text)
print(res)
```

```
regex x
['P', 't', 'h', 'n']
```

^ и **\$** — Начало и конец строки

```
import re

text = 'Это просто текст'
res1 = re.findall(r'^Это', text)
res2 = re.findall(r'текст$', text)
print(res1, res2, sep='\n')
```

```
regex x
['Это']
['текст']
```

a|b — Соответствует a или b

```
import re

text = 'В доме 8 дробь 16 кот живёт'
res = re.findall(r'[1|8]', text)
print(res)
```

```
regex x
['8', '1']
```

(...) — Группирует выражение и возвращает найденный текст

```
import re

text = 'Сложение 5+9=14, вычитание 40-5=35'
res1 = re.findall(r'(\d[^+=])', text)
res2 = re.findall(r'(\d[+=])', text)
print(res1, res2, sep='\n')
```

```
regex x
['14', '40', '35']
['5+', '9=', '5=']
```

На картинке выше мы сгруппировали следующие запросы – показать любую цифру и справа добавили показать всё кроме + и =. В итоге вывелись только цифры у которых справа нет этих символов.

Ниже запросы – показать любую цифру и справа добавили показать всё с + и =. В результате вывелись цифры имеющие справа + и =.

```
import re

text = 'Сложение 5+9=14, вычитание 40-5=35'
res = re.findall(r'([a-я][e])', text)
print(res)
```

```
regex x
['же', 'ие', 'ие']
```

В этом примере мы выводим все буквы от **а** до **я** и добавляем поиск только буквы **е**. В результате выводятся все символы имеющие справа букву **е**.